

acs.py

```
1  # ACS.py
2
3  class ACSclass:
4      def __init__(self, alpha=1.0, c0=1.0, enabled = True):
5          self.enabled = enabled
6          self.alpha = float(alpha)
7          self.c = float(c0)
8
9          self.c_old = None
10         self.c_oldold = None
11
12         self.iter = 0
13
14     def update(self, g, gmax):
15
16         # ---- ACS OFF ----
17         if not self.enabled:
18             self.c = 1.0
19             return self.c
20
21         # Safety
22         g = max(float(g), 1e-12)
23         gmax = float(gmax)
24
25         if self.iter == 0:
26             self.c = gmax / g
27             self.alpha = 1.0
28
29         elif self.iter == 1:
30             self.c_old = self.c
31             self.c = gmax / g
32
33         else:
34             self.c_oldold = self.c_old
35             self.c_old = self.c
36
37             scale_up = 1.20
38             scale_down = 0.80
39             decimal = 2
40
41             ratio = gmax / g
42
43             c_new = self.alpha * ratio + (1 - self.alpha) * self.c_old
44
45             c_oldold_r = round(self.c_oldold, decimal)
46             c_old_r = round(self.c_old, decimal)
47             c_new_r = round(c_new, decimal)
48
49             oscillating = (
50                 (c_oldold_r < c_old_r and c_old_r > c_new_r) or
51                 (c_oldold_r > c_old_r and c_old_r < c_new_r)
52             )
53
54             if oscillating:
55                 self.alpha = max(0.5, self.alpha * scale_down)
56             else:
57                 self.alpha = min(1.0, self.alpha * scale_up)
58
59             self.c = self.alpha * ratio + (1 - self.alpha) * self.c_old
60
61         self.iter += 1
62
```

acs.py

```
63     print(  
64         f"[ACS] iter={self.iter:3d} | "  
65         f"g={g:.4f} | gmax={gmax:.4f} | "  
66         f"alpha={self.alpha:.3f} | c={self.c:.4f}"  
67     )  
68  
69     return self.c
```